

exact_diagonalization_cpp: a five-axis C++/CUDA/MPI toolkit for exact diagonalization of quantum lattice models

Zhengbang Zhou^{1*}

¹ Institution to be filled in

* zhengbang@example.org

Abstract

We present `exact_diagonalization_cpp`, a research-grade C++17/CUDA/MPI toolkit for exact diagonalization (ED) of quantum spin Hamiltonians, exposed through a single static C++ library, a single command-line driver, and a pybind11-backed Python package (`quantum_ed`). The defining design choice is a *five-axis solver organization*: every supported workload is specified by a tuple (`solver`, `fixed_sz`, `gpu`, `mpi`, `symm`) on a flat `EDParameters` bag rather than by a hand-tangled enum. This collapses a previously combinatorial set of entry points onto one canonical dispatcher per backend and makes every Krylov-family algorithm composable with each of GPU acceleration, MPI-parallel halo exchange, fixed- S_z basis restriction, and streaming space-group projection. We describe the algorithms (the Lanczos family, Krylov–Schur, Davidson, LOBPCG, ARPACK, ScaLAPACK, FTLM/LTLM/Hybrid, microcanonical and canonical TPQ, dynamical structure factor) at an interface level, the cross-cutting performance hygiene that keeps the dispatcher’s overhead from dominating small problems (output gating, thread budgeting, swap rotations), and the GPU/MPI hot-path optimizations that brought the distributed Lanczos and the GPU Lanczos to bandwidth-limited operation (batched cgs2 re-orthogonalization with two `MPI_Allreduce` per iteration, persistent halo and ring-buffer buffers, device-pointer-mode cuBLAS scalars, an automatic ScaLAPACK block-size heuristic). On a 16-thread x86 reference workstation, the CPU Lanczos runs $\sim 1.5\text{--}10\times 10^3$ faster than `scipy.sparse.linalg.eigsh` backed by QuSpin’s matrix construction at $d \in [4096, 262,144]$, and faster than `XDiag.eigval0` at every size we tested both with and without S_z conservation. The package is released under the MIT license and ships with 150 C++ Catch2 tests and 197 Python tests in CI.

Copyright Z. Zhou.

This work is licensed under the
Creative Commons
Attribution 4.0 International
License.
Published by the SciPost
Foundation.

Received Date
Accepted Date

Published Date

1	Contents	
2	1 Introduction	3
3	2 Five-axis solver organization	4
4	2.1 Axis 1: SOLVER	4
5	2.2 Axis 2: FIXED_SZ	4
6	2.3 Axis 3: GPU	4
7	2.4 Axis 4: MPI	5
8	2.5 Axis 5: SYMM	5
9	2.6 Capability matrix	5
10	3 Methods reachable through the five-axis dispatcher	5
11	3.1 The Lanczos family	5
12	3.2 Restart-based Krylov: Krylov–Schur and ARPACK	6
13	3.3 Davidson and LOBPCG	6
14	3.4 Spectral slicing and shift-invert	6
15	3.5 Dense diagonalisation	7
16	3.6 Finite-temperature: FTLM, LTLM, Hybrid	7
17	3.7 Thermal Pure Quantum states	7
18	3.8 Dynamical and static structure factors	7
19	4 Cross-cutting performance optimizations	7
20	4.1 HDF5 output gating	8
21	4.2 OpenMP / OpenBLAS thread budgeting	8
22	4.3 Vector swap-rotates	8
23	4.4 Persistent halo and ring-buffer staging in MPI/GPU operators	8
24	4.5 Device-pointer-mode cuBLAS scalars	9
25	4.6 Batched CGS2 reorthogonalisation in distributed Lanczos	9
26	4.7 ScaLAPACK auto-block-size	9
27	5 Usage	10
28	5.1 Python: single-call dispatcher	10
29	5.2 C++ static library	10
30	5.3 CLI	11
31	6 Validation and benchmarks	11
32	6.1 Test suite	11
33	6.2 Cross-library validation	11
34	6.3 Single-node CPU benchmarks (reproducible on a workstation)	12
35	6.4 Figures requiring runs not attainable on a local workstation	12
36	7 Discussion and conclusions	17
37	References	19

1 Introduction

Exact diagonalization (ED) is the only family of methods for quantum lattice models that returns numerically exact eigenvalues, eigenvectors and finite-temperature expectation values of a chosen Hamiltonian without any uncontrolled approximation. Its only drawback is the exponential growth of the Hilbert space with the number of sites, which has motivated four decades of careful coding around symmetry projection, hashing of basis states, sparse matrix-vector products [1–5], finite-temperature samplers [6–10], and modern restart-based Krylov solvers [11–14]. Existing public packages such as QuSpin [15, 16], $\mathcal{H}\Phi$ [17], ALPS [18], and XDiag [19] cover slices of this design space, but in our experience either limit the user to a fixed front-end (Python over a C kernel, in QuSpin’s case), require recompilation per algorithmic axis, or do not expose distributed-memory and GPU paths to the same in-process API.

This paper presents `exact_diagonalization_cpp`, a single toolkit that addresses three pain points we hit repeatedly when using the existing tools for production research on frustrated magnets:

1. **The Python-driver overhead barrier.** For small-to-moderate Hilbert dimensions ($d \lesssim 10^5$), the wall time of `scipy.sparse.linalg.eigsh` on a QuSpin Hamiltonian is dominated by per-iteration Python dispatch, on-the-fly basis state recomputation, and HDF5 traffic from the wrapper layer rather than by the matrix-vector product itself. We therefore re-engineered the SpMV hot path against an explicit precomputed sparse representation in C++17, with all per-iteration bookkeeping in compiled code, and find 3–4 orders of magnitude speedups in this regime (see Section 4 and Refs. [15, 19]).
2. **The combinatorial-axis explosion of solvers.** A production project that wants *Lanczos with full reorthogonalization, on the $S_z = 0$ sector, on a single GPU, with space-group projection* needs a different code path in most existing codebases than *Lanczos with full reorthogonalization, on the full Hilbert space, distributed across MPI ranks, without projection*. In our pre-release internal versions we had separate enum entries for `LANCZOS_FIXED_SZ`, `LANCZOS_GPU`, `LANCZOS_GPU_FIXED_SZ`, `LANCZOS_MPI`, `LANCZOS_SYMM...` which made it hard to compose more than two axes cleanly. We replace this with five orthogonal flags (Section 2) on a flat parameter struct, which the dispatcher canonicalizes into the appropriate kernel.
3. **Distributed and GPU paths that share the API.** Most ED toolkits bolt on distributed-memory or GPU code as separate executables. We make the five-axis dispatcher routable across all backends from the same in-process API: `distributed_lanczos`, `distributed_ftlm`, `distributed_tpq`, the NCCL-based `distributed_lanczos_gpu`, and the per-sector `GPUSymmetrizedOperator` are all visible through a single Python entry point.

The contributions of this codebase are therefore (i) the five-axis dispatcher contract and its implementation across ~ 30 backends, (ii) a CPU SpMV+Lanczos that beats the standard `scipy.sparse.linalg.eigsh`+QuSpin pairing by orders of magnitude in the small/medium regime where ED for production condensed-matter studies actually lives, and (iii) the cross-cutting performance optimizations (HDF5 output gating, OpenMP/OpenBLAS thread budgeting, swap rotations, batched CGS2 reorthogonalization, persistent GPU/MPI buffers, device-pointer-mode cuBLAS, ScaLAPACK auto-block-size) that keep that headline number from collapsing on the distributed-memory and GPU paths.

The rest of the paper is organized as follows. Section 2 defines the five-axis solver organization. Section 3 surveys the algorithms reachable through that contract. Section 4 catalogues the cross-cutting performance optimizations. Section 5 walks through the CLI, C++ and Python interfaces. Section 6 reports validation and benchmark results, including the protocols required to reproduce the distributed-scale figures that are not attainable on a single workstation. Section 7 concludes.

2 Five-axis solver organization

The five orthogonal axes are

$$(\text{SOLVER}, \text{FIXED_SZ}, \text{GPU}, \text{MPI}, \text{SYMM}). \quad (1)$$

Each axis is independent of the others up to two well-documented exceptions: dense kernels (`FULL`, `SCALAPACK[_MIXED]`) cannot be combined with on-the-fly symmetry projection because they need an explicit dense block matrix, and `LOBPCG_GPU` currently aliases to the GPU Davidson kernel (a deliberate indirection while a dedicated GPU LOBPCG is being prepared).

2.1 Axis 1: solver

The `DiagonalizationMethod` enum names the *algorithm* only — not the backend. The currently shipped values are `LANCZOS`, `LANCZOS_SELECTIVE`, `LANCZOS_NO_ORTHO`, `BLOCK_LANCZOS`, `KRYLOV_SCHUR`, `BLOCK_KRYLOV_SCHUR`, `DAVIDSON`, `LOBPCG`, `CHEBYSHEV_FILTERED`, `SHIFT_INVERT[_ROBUST]`, `IMPLICIT_RESTART_LANCZOS`, `THICK_RESTART_LANCZOS`, `BICG`, `ARPACK_SM`, `ARPACK_LM`, `ARPACK_SHIFT_INVERT`, `ARPACK_ADVANCED`, `FULL`, `OSS`, `SCALAPACK`, `SCALAPACK_MIXED`, `FTLM`, `LTLM`, `HYBRID`, `mTPQ`, and `cTPQ`.

2.2 Axis 2: fixed_sz

Setting `EDParameters::use_fixed_sz=true` (with `params.n_up` naming the desired sector) restricts the basis from the full Hilbert space $\text{dim} = 2^N$ to the combinatorial S^z -conserved subspace $\text{dim} = \binom{N}{N_\uparrow}$, using the standard tagged binomial indexing of [1]. The same kernel that runs in the full Hilbert space then runs on the smaller basis with no further user intervention. Dense kernels (`FULL`, `SCALAPACK`) auto-detect S_z conservation in the Hamiltonian and split into sectors when the user does not pass an explicit `n_up`; explicit sector restriction is honoured when `use_fixed_sz` is set.

2.3 Axis 3: gpu

Setting `use_gpu=true` routes through the matching `GPUOperator` matrix-free SpMV, cuBLAS Rayleigh–Ritz, and cuSOLVER-backed dense diagonalisation paths [20–22]. Six iterative solvers (`LANCZOS`, `BLOCK_LANCZOS`, `KRYLOV_SCHUR`, `BLOCK_KRYLOV_SCHUR`, `DAVIDSON`, `LOBPCG`) and the dense `FULL` solver have a GPU port. Finite-temperature methods (`FTLM`, `mTPQ`, `cTPQ`) inherit the GPU Lanczos path. The seven remaining CPU-only iterative methods are intentional: shift-invert needs a GPU sparse direct factoriser that does not yet exist upstream, IRL/TRL/BICG are superseded by Krylov–Schur whose GPU port is shipped, and ARPACK is a Fortran library with no drop-in GPU replacement.

2.4 Axis 4: mpi

The MPI surface uses 1D row partitioning of the state vector with an `MPI_Alltoallv` halo exchange for the off-rank action of H , plus an orbit-balanced partition for the symmetry-projected case. Three workhorse drivers are exposed: `distributed_lanczos` (and a templated kernel reused by `distributed_lanczos_symmetry`), `distributed_ftlm`, and `distributed_tpq` (canonical TPQ). A multi-GPU path `distributed_lanczos_gpu` keeps the state vector resident on each rank's GPU and does the halo through NCCL [23]. Setting `use_mpi=true` on the in-process dispatcher emits a hard error pointing at the dedicated `ed_distributed_main` binary or the `quantum_ed.mpi.run_distributed` launcher; this is by design because in-process MPI launches conflict with most Python kernels.

2.5 Axis 5: symm

Setting `use_symmetry=true` routes through the canonical *streaming* symmetry kernel. A `SymmetryGroupInfo` object, built either programmatically from the `quantum_ed.symmetry` DSL (`translation`, `reflection_1d`, `site_swap`, `compose`, `power`, `generate_group`) or read from disk, is attached to the operator. The kernel then iterates orbit representatives without materialising the full sector basis on disk. Streaming symmetry composes with GPU (per-sector kernels in `gpu_symmetrized_operator.cu`) and with `FIXED_SZ`. Eight pre-existing symmetry entry points (explicit-block, chunked-disk, fixed-Sz-symmetrized, etc.) were collapsed during the Phase 7.1 refactor onto this single canonical path; only two deliberately separate escape hatches remain (`--chunked-symm` and `--disk-streaming`) for memory-budget edge cases at very large N where the user wants explicit on-disk sector materialisation.

2.6 Capability matrix

Table 1 summarises which algorithm $\times$ backend combinations are presently shipped ($\checkmark$) and which have a deliberate negative answer ($-$). The table is automatically checked against the dispatcher in CI: every ($\checkmark$) cell has at least one Catch2 or pytest case that exercises it.

3 Methods reachable through the five-axis dispatcher

We summarise here, at an interface level, the algorithms behind each solver token. References point to the canonical paper; the headers implementing each algorithm are listed in `include/ed/solvers/` and `include/ed/distributed/`.

3.1 The Lanczos family

The reference Lanczos three-term recurrence [14,24] is implemented in `Lanczos.h` with full reorthogonalisation against a stored basis (rotated through a fixed-size ring buffer when the basis exceeds a configurable cap, after which it falls back to selective reorthogonalisation [25]). The CPU path uses `std::swap` rotation between the three active vectors so that the per-iteration vector traffic is one local SpMV plus $\mathcal{O}(d)$ axpy for the recurrence, never an $\mathcal{O}(d)$ `std::copy` of a full Lanczos vector.

The block variant `block_lanczos` solves $HV_k = V_k T_k + V_{k+1} B_{k+1}^\top$ for a block size taken from `params.block_size`, using a Householder QR for the inner B_{k+1} and full reorthogonalisation against the previous block vectors. Block Lanczos is the recommended

Table 1: Capability matrix of the five-axis dispatcher. Every ($\checkmark$) cell has at least one regression test in the shipped 150 + 197 test suite. LOBPCG_GPU is currently implemented by aliasing to the GPU Davidson kernel.

Solver	CPU	GPU	MPI	fixed-Sz	symm
LANCZOS	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$
LANCZOS_SELECTIVE	$\checkmark$	–	$\checkmark$	$\checkmark$	$\checkmark$
LANCZOS_NO_ORTHO	$\checkmark$	–	$\checkmark$	$\checkmark$	$\checkmark$
BLOCK_LANCZOS	$\checkmark$	$\checkmark$	–	$\checkmark$	$\checkmark$
KRYLOV_SCHUR	$\checkmark$	$\checkmark$	–	$\checkmark$	$\checkmark$
BLOCK_KRYLOV_SCHUR	$\checkmark$	$\checkmark$	–	$\checkmark$	$\checkmark$
DAVIDSON	$\checkmark$	$\checkmark$	–	$\checkmark$	$\checkmark$
LOBPCG	$\checkmark$	$\checkmark^\dagger$	–	$\checkmark$	$\checkmark$
CHEBYSHEV_FILTERED	$\checkmark$	–	–	$\checkmark$	–
SHIFT_INVERT[_ROBUST]	$\checkmark$	–	–	$\checkmark$	–
IRL / TRL / BICG	$\checkmark$	–	–	$\checkmark$	–
ARPACK_*	$\checkmark$	–	–	$\checkmark$	$\checkmark$
FULL / OSS	$\checkmark$	$\checkmark^\ddagger$	–	$\checkmark^\ddagger$	–
SCALAPACK[_MIXED]	–	–	$\checkmark$	$\checkmark^\ddagger$	–
FTLM	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$
LTLM / HYBRID	$\checkmark$	–	–	$\checkmark$	$\checkmark$
mTPQ	$\checkmark$	$\checkmark$	–	$\checkmark$	$\checkmark$
cTPQ	$\checkmark$	$\checkmark$	$\checkmark^\S$	$\checkmark$	$\checkmark$

† LOBPCG_GPU aliases the GPU Davidson kernel. ‡ Dense kernels auto-split Sz sectors when applicable. § MPI canonical TPQ via `ed::distributed::distributed_tpq`; microcanonical mTPQ has no MPI driver (use sample-parallel CPU runs).

162 path for finding several lowest eigenpairs simultaneously, and is the underlying GPU kernel
 163 for the multi-eigenvalue API.

164 3.2 Restart-based Krylov: Krylov–Schur and ARPACK

165 The Krylov–Schur algorithm of Stewart [12] is implemented in `krylov_schur.h` with both
 166 single and block restart, on CPU and GPU. `ARPACK_*` wraps the Fortran ARPACK [11] via
 167 `arpack/arpack_*.h` for strict regression parity with legacy production runs; the `ARPACK_ADVANCED`
 168 flavour exposes shift-invert with auto-tuned shift selection. In greenfield code, our rec-
 169 ommendation is `KRYLOV_SCHUR` or `BLOCK_KRYLOV_SCHUR` since they have a working GPU
 170 port.

171 3.3 Davidson and LOBPCG

172 `DAVIDSON` (header `davidson.h`) implements the diagonal-preconditioned Davidson iter-
 173 ation [26] with a maximum-subspace cap (`max_subspace` on `EDParameters`). `LOBPCG`
 174 (header `lobpcg.h`) implements the Locally-Optimal Block PCG of Knyazev [27]; the host
 175 Rayleigh–Ritz step is wrapped in the Phase 8 `ThreadBudgetScope` (see Section 4).

176 3.4 Spectral slicing and shift-invert

177 `CHEBYSHEV_FILTERED` (header `chebyshev_filtered.h`) implements an interior-eigenvalue
 178 spectral filter: given a target window $[\lambda_-, \lambda_+]$ (`params.target_lower/upper`), a Cheby-
 179 shev polynomial in H is applied to a starting subspace and the result is then refined with
 180 Lanczos. `SHIFT_INVERT` (header `shift_invert.h`) is the textbook shift-and-invert Lanc-

zos with a single sparse LU per shift; the `SHIFT_INVERT_ROBUST` variant adds adaptive shift restarting. These two methods are CPU-only; the gating dependency is a production-grade complex-Hermitian sparse direct solver on GPU, which is not yet upstream in cuSPARSE.

3.5 Dense diagonalisation

`FULL` (header `full_diagonalization.h`) routes through LAPACK `zheevd/zheev` [28] on CPU and `cuSOLVER zheevd` [22] on GPU. `SCALAPACK` and `SCALAPACK_MIXED` (header `scalapack_diag.h`) wrap `pzheevr/pzheevd` of ScaLAPACK [29,30], with a mixed-precision iterative refinement loop in the `MIXED` variant.

3.6 Finite-temperature: FTLM, LTLM, Hybrid

The Finite-Temperature Lanczos Method (FTLM) [6] computes $\langle O \rangle_T = Z^{-1} \sum_r \langle r | e^{-\beta H/2} O e^{-\beta H/2} | r \rangle$ with $|r\rangle$ a random vector and the matrix exponentials evaluated in the Lanczos basis it builds against $H|r\rangle$. The Low-Temperature Lanczos Method (LTLM) [7] uses a ground-state-projected stochastic recipe that converges faster at low T but at higher cost per sample. Our `HYBRID` method auto-crosses over from LTLM at low T to FTLM at high T at a user-controllable boundary.

3.7 Thermal Pure Quantum states

The TPQ family contains two physically distinct algorithms that share the name; we keep them as separate enum tokens. The *microcanonical* mTPQ of Hams & De Raedt / Sugiura–Shimizu [8,9] iterates $|k+1\rangle = (\Lambda - H)|k\rangle$ from a random $|0\rangle$ with Λ chosen as `LargeValue`, then renormalises and reads $\beta(k)$ off from $\langle H \rangle$ a posteriori. The *canonical* cTPQ [10] Taylor-evolves $e^{-\beta H/2}|r\rangle$ in $\delta\beta$ substeps, returning $\langle O \rangle$ at a fixed schedule of β values. The distributed `ed::distributed::distributed_tpq` implements the canonical recipe across MPI ranks (Section 2.4); a microcanonical MPI driver is not exposed because in our experience sample-parallel CPU runs of mTPQ saturate the network anyway.

3.8 Dynamical and static structure factors

The DSSF/SSSF surface (`src/dssf/`, CLI subcommand `./ED dssf`) supports three workflows: `ground_state_dssf` (Lanczos continued-fraction at $T = 0$), `static_thermal` (FTLM-evaluated equal-time correlators), and `dynamical_thermal` (FTLM-evaluated continued-fraction $S(\mathbf{q}, \omega; T)$). The continued-fraction evaluation is the standard Mori projection construction [31,32]. The GPU dispatcher `runGPUDynamicalResponse[Thermal]` reuses the GPU Lanczos basis emitted by `LANCZOS_GPU`; a fully distributed DSSF is gated on the deferred TPQ-DSSF Mori continued-fraction work.

4 Cross-cutting performance optimizations

The five-axis dispatcher only delivers production performance if its overhead does not eclipse the per-iteration kernel cost. Two generations of optimizations (Phase 6.1 on CPU; Phase 8 on GPU and MPI) bring every backend to a regime where the dominant cost is either the matrix-free SpMV or the dense Rayleigh–Ritz, with neither HDF5 nor allocator nor pthread-creation traffic visible in profiles.

4.1 HDF5 output gating

The legacy CPU solvers wrote `ed_results.h5` on every invocation. For the small-to-medium SpMV regime ($d \lesssim 10^5$) this single file open dominated wall time on fast filesystems and was catastrophic on networked home directories. We added `HDF5IO::isDisabledOutputPath(path)` a predicate that recognises "", `"/dev/null"`, and a small set of sentinel paths and short-circuits all write paths through it. Every `HDF5IO` entry point (`createOrOpenFile`, `saveEigenvalues`, `saveEigenvector`, `saveDiagonalizationResults`, `ensureTPQSampleGroup`, `saveTPQState`, all `saveTPQ*` / `saveFTLM*` / `saveStaticResponse` / `saveHybridThermalResults` / `ensureFTLMSampleGroups` / `ensureTimeCorrelationGroups` / `fileExists` entry points) consults the predicate first. In Python, the `lanczos()` / `full_diagonalization()` / `finite_temperature_lanczos()` / `low_temperature_lanczos()` / `hybrid_thermal_method()` wrappers default `output_dir=""`; the high-level `exact_diagonalization_core(op, method, params)` dispatcher remaps an empty `params.output_dir` to `"/dev/null"` before fanning out to any backend. Passing `output_dir="."` restores the legacy on-disk dump, byte-for-byte identical to the pre-Phase-6 output.

4.2 OpenMP / OpenBLAS thread budgeting

Capping the OpenMP+OpenBLAS pthread pool to $\min(\text{nproc}, \lceil \log_2 d \rceil)$ at the start of each solver call (and restoring it at scope exit) consistently improves small-problem performance. The `ed::parallel::ThreadBudgetScope` RAI type implements this cap. Phase 6.1 wrapped every CPU iterative entry point. Phase 8 extended this to MPI host code (`distributed_lanczos`, `distributed_lanczos_kernel`, `distributed_tpq`, `distributed_ftlm`; sized against the rank-local slab, not the global dimension) and GPU host code (`GPULanczos::solveTridiagonal`, `GPUBlockLanczos::solveBlockTridiagonal`, `lobpcg.solve_generalized_eigenproblem`, `GPUFTLMSolver::run`). The cap can be disabled with `ED_AUTO_THREADS=0` for users who already pin threads with `mpiexec --bind-to`.

4.3 Vector swap-rotates

The Lanczos / Chebyshev / shift-invert inner loops rotate three active vectors $(v_{i-1}, v_i, v_{i+1}) \rightarrow (v_i, v_{i+1}, v_{i+2})$ on every iteration. Pre-Phase-6 these rotations were three `std::copy` of $\mathcal{O}(d)$ bytes. The current path uses `std::swap(v_old, v_cur); std::swap(v_cur, v_new);` which is $\mathcal{O}(1)$ pointer surgery on `std::vector`. The underlying allocations are kept alive by the surrounding scope, so the swap is safe.

4.4 Persistent halo and ring-buffer staging in MPI/GPU operators

Pre-Phase-8 `DistributedOperator::apply()` and `DistributedSymmetryOperator::apply()` allocated `std::vector<std::complex<double>>` send / recv / halo buffers on every matvec — two heap round-trips per matvec, plus first-touch faulting if the allocator handed back fresh pages. Both operators now own mutable `std::vector<Complex>` `send_buf_`, `recv_buf_`, `halo_buf_` sized exactly once in `build_comm_pattern_` / the constructor, and `apply()` no longer allocates in the hot path.

The GPU Lanczos has the same problem at the host-to-device-pointer-table level: `GPULanczos::orthogonalize()` used to rebuild the windowed pointer table of basis vectors (`num_check` device pointers, 8 B each) on the host every iteration and `cudaMemcpy` it to the persistent `d.ortho_basis_ptrs_` device array — one synchronous default-stream H2D copy per Lanczos iteration. We added a persistent `d.ortho_basis_ptrs_full_`, a device-side mirror of the *entire* ring-buffer pointer table, populated once at `allocateMemory()`

time. In the common case ($\text{iter} \leq \text{num_stored_vectors_}$, no ring-buffer wrap), the orthogonalise kernel reads from `d_ortho_basis_ptrs_full_` directly with `num_vecs = num_check`; H2D traffic in the hot path is zero. The wrapped windowed-reorth case retains the legacy host→device copy.

4.5 Device-pointer-mode cuBLAS scalars

In the GPU Lanczos hot path we compute $\alpha = \langle v | H | v \rangle$ via `cublasZdotc(...)`. Pre-Phase-8 this used `CUBLAS_POINTER_MODE_HOST`, which forces an implicit device→host synchronisation per iteration; the follow-up $w \leftarrow w - \alpha v$ on the same stream cannot issue until that sync completes. We added device-resident scalar buffers `d_alpha_dev_`, `d_neg_alpha_dev_`, a pinned host slot `h_alpha_pinned_`, and a one-thread kernel `negateZScalarRealKernel` that writes $-\Re(*src)$ into the destination, preserving bit-identity with the previous host-side cast. The dot now runs in `CUBLAS_POINTER_MODE_DEVICE` and the `negate+axpy` chain runs back-to-back without host involvement. α is then async-copied to `h_alpha_pinned_` and consumed only after `vectorNorm(d_w_)`, which implicitly drains the stream because the convergence check on β is host-side.

4.6 Batched CGS2 reorthogonalisation in distributed Lanczos

`distributed_lanczos` and the templated `distributed_lanczos_kernel` (also used by `DistributedSymmetryOperator`) used to run modified Gram–Schmidt reorthogonalisation:

$$\text{for } k = 0 \dots m-1: \quad c_k = \langle V_k | w \rangle \quad [1 \text{ MPI_Allreduce}], \quad w \leftarrow w - c_k V_k.$$

For full-reorth Lanczos at iteration m that is m serial `MPI_Allreduce` calls; over a 200-iteration run, $\sim 20\,000$ small network round-trips that dominate the local SpMV at scale. We swapped MGS for classical Gram–Schmidt run twice [33, 34]:

$$\text{pass 1: } c_k^{\text{local}} = \langle V_k | w \rangle_{\text{local}} \forall k, \text{ 1 batched MPI_Allreduce, } w \leftarrow w - \sum_k c_k V_k$$

pass 2: (same again — “twice is enough”)

A helper `dist_zdotc_batched` packs the m local complex dot products into a flat $2m$ -double buffer and Allreduces them in one shot (we do not reduce on `std::complex<double>` directly because `MPI_C_DOUBLE_COMPLEX` with `MPI_SUM` is implementation-defined in some MPI 3.x stacks [35]). Total network traffic: 2 batched Allreduces per Lanczos iteration regardless of m , instead of m scalar Allreduces. For tiny $m < 8$ we keep the MGS path because the second CGS pass is then pure overhead.

4.7 ScaLAPACK auto-block-size

The legacy ScaLAPACK default $m_b = n_b = 64$ over-blocks small matrices and under-blocks the larger ones. We added a flag `scalapack_block_size_auto` (default `true`) on `EDParameters`, `EDConfig::DiagonalizationConfig`, and the Python binding. When auto is on, the solver replaces the user’s m_b/n_b with `GETOPTIMALBLOCKSIZE(N, NPROW, NPCOL)` at solve time — a dimension-and-grid-aware heuristic that balances local tile size against the BLACS process grid for the actual matrix dimension. The explicit CLI `--scalapack-block-size N` sets the value *and* flips auto off, so legacy callers tuning the block size manually are not surprised.

5 Usage

The toolkit is reachable through three coherent surfaces: a CLI binary `./ED`, a C++17 static library `libexact_diagonalization.a`, and a Python package `quantum_ed` built via `scikit-build-core + pybind11`. We illustrate the most common entry point in each.

5.1 Python: single-call dispatcher

The recommended entry point for new code is the Phase-5 unified dispatcher `exact_diagonalization_core`, which composes all five axes through `EDParameters`. The example below constructs a 12-site Heisenberg chain through the in-process `quantum_ed.input` builder, then runs Krylov–Schur on the $S^z = 0$ sector with translation + reflection symmetry projection on GPU. This is one process and a single Python interpreter; no shell glue, no `InterAll.dat` round-trip.

```
import quantum_ed as qed

lat = qed.input.lattice.chain(12, pbc=True)
op = (qed.input.HamiltonianBuilder(lat.num_sites)
      .heisenberg(lat.nn_pairs(), 1.0)
      .to_operator())

info = qed.symmetry.group_from_generators(
    12, [qed.symmetry.translation(12),
         qed.symmetry.reflection_1d(12)],
    sector_quantum_numbers=[0, 0],
)
op.set_symmetry_info_from_dict(info)

p = qed.EDParameters()
p.num_eigenvalues = 4
p.tolerance = 1e-12
p.use_fixed_sz = True; p.n_up = 6
p.use_gpu = qed.has_cuda_build()
p.use_symmetry = True

res = qed.exact_diagonalization_core(
    op, qed.DiagonalizationMethod.KRYLOV_SCHUR, p,
)
print("E0..E3 =", sorted(res.eigenvalues)[:4])
```

For distributed runs, the same operator and parameter struct are serialised through a directory and a launcher subprocess.

```
qed.input.HamiltonianBuilder(lat.num_sites) \
    .heisenberg(lat.nn_pairs(), 1.0) \
    .write_directory("./chain12", lattice=lat)

if qed.has_mpi_build():
    qed.mpi.run_distributed("./chain12",
                           method="lanczos",
                           n_ranks=8)
```

5.2 C++ static library

The same five-axis call from C++:

```
#include <ed/core/ed_dispatch_symmetry.h>
#include <ed/core/operator.h>
```

```

355
356 auto op = std::make_shared<Operator>(<math>N=12</math>);
357 op->loadFromInterAllFile("InterAll.dat");
358
359 EDParameters p;
360 p.num_eigenvalues = 4;
361 p.tolerance = 1e-12;
362 p.use_fixed_sz = true; p.n_up = 6;
363 p.use_gpu = true;
364 p.use_symmetry = true;
365
366 auto res = ed_dispatch::exact_diagonalization_from_directory(
367     "/chain12", DiagonalizationMethod::KRYLOV_SCHUR, p);
368 std::cout << "E0 = " << res.eigenvalues[0] << "\n";
369

```

370 5.3 CLI

371 Every flag on EDParameters is exposed through ./ED. The Phase 8 ScaLAPACK auto-
372 block-size and explicit override are wired through:

```

373
374 ./ED ./chain24 --method=SCALAPACK \
375     --eigenvalues=8 \
376     --scalapack-nprow=4 --scalapack-npcol=4 \
377     --scalapack-block-size-auto # default
378 ./ED ./chain24 --method=SCALAPACK \
379     --scalapack-block-size=128 # implicit auto-off
380
381 mpiexec -n 16 ./ED ./chain24 --method=LANCZOS_MPI \
382     --symm --tolerance=1e-12
383

```

384 6 Validation and benchmarks

385 6.1 Test suite

386 The package ships with 150 C++ Catch2 v3 tests (tests/unit/, tests/integration/)
387 and 197 Python pytest tests (python/tests/). Every (✓) cell in Table 1 has at least one
388 regression test that exercises it; in particular the canonicalisation of legacy solver-name
389 aliases through ed::canonicalize_method_and_flags onto the five-axis tuple is covered
390 by tests/unit/test_method_canonicalize.cpp and python/tests/test_dispatcher.py.

391 6.2 Cross-library validation

392 We validate against two independent reference implementations on the canonical 1D pe-
393 riodic Heisenberg chain at $S = 1/2$ with $J = 1$:

- 394 1. **XDiag** [19]: ground-state energies recovered to $\sim 10^{-12}$ for $N \in \{12, 14, 16, 18, 20\}$ in
395 both the full Hilbert space and the $S^z = 0$ sector. Reproducer: benchmarks/bench_vs_xdiag.py
396 + benchmarks/bench_vs_xdiag.jl.
- 397 2. **QuSpin** + **scipy** [15, 16]: SpMV and ground-state Lanczos cross-checks at $d \in$
398 $\{4096, \dots, 1\,048\,576\}$, energies agree to $\sim 10^{-12}$. Reproducer: benchmarks/bench_vs_quspin.py.

Table 2: Head-to-head against XDiag [19] on the 1D periodic Heisenberg chain at $J = 1$, $S = 1/2$, full Hilbert space. `tol = 1e-10` on the `quantum.ed` side, library default on the XDiag side. 16 threads, OpenBLAS, x86_64 WSL2/gcc-13; `quantum.ed.lanczos` default `output_dir` (HDF5 off).

N	dim	qed apply	XDiag apply	qed Lanczos	XDiag Lanczos	speedup
12	4,096	34.1 μ s	483.8 μ s	1.8 ms	12.7 ms	6.9 $\times$
14	16,384	86.6 μ s	701.3 μ s	2.0 ms	132.7 ms	65.9 $\times$
16	65,536	6.1 ms	10.5 ms	4.2 ms	196.8 ms	46.9 $\times$
18	262,144	6.6 ms	9.1 ms	23.8 ms	709.9 ms	29.8 $\times$
20	1,048,576	10.8 ms	30.7 ms	233.1 ms	2.54 s	10.9 $\times$

Table 3: Same head-to-head as Table 2 restricted to the $S^z = 0$ sector (XDiag.Spinhalf(N , $N/2$) vs. our FixedSzOperator). This is the configuration XDiag is most heavily tuned for [19].

N	dim	qed apply	XDiag apply	qed Lanczos	XDiag Lanczos	speedup
12	924	8.1 μ s	547.6 μ s	1.2 ms	10.6 ms	9.2 $\times$
14	3,432	27.7 μ s	448.1 μ s	2.1 ms	18.7 ms	8.9 $\times$
16	12,870	92.0 μ s	685.8 μ s	3.7 ms	42.0 ms	11.3 $\times$
18	48,620	511.5 μ s	2.93 ms	26.5 ms	121.1 ms	4.6 $\times$
20	184,756	905.7 μ s	10.25 ms	252.2 ms	494.2 ms	2.0 $\times$

6.3 Single-node CPU benchmarks (reproducible on a workstation)

On a 16-thread x86_64 reference (WSL2 Ubuntu, gcc-13, OpenBLAS, HDF5 disabled by default in the Python wrapper) the head-to-head against XDiag at `tol = 1e-10` on the full Hilbert space gives the wall-times in Table 2; the $S^z = 0$ -sector counterpart is in Table 3. Both agree on E_0 to $\sim 10^{-12}$. The complete reproducer command is

```
PYTHONPATH=python python3 benchmarks/bench_vs_xdiag.py \
  --sizes 12 14 16 18 20 \
  --threads $(nproc) \
  --output docs/benchmarks/bench_vs_xdiag.json
```

The QuSpin head-to-head at the same dimensions gives `eigsh/quantum.ed.lanczos` ratios of order 1.5×10^3 to 9.7×10^3 at `tol = 1e-10`; since QuSpin's `eigsh` path delegates to scipy's ARPACK wrapper, this measures the combined overhead of Python dispatch and the QuSpin matrix-construction layer plus the ARPACK iteration cost, not ARPACK in isolation.

6.4 Figures requiring runs not attainable on a local workstation

Three categories of figures characterise the toolkit at distributed and very-large- N scale that we do not produce on a laptop. Each required-run callout below states the exact command, hardware, expected wall-clock, and the data files the postprocessor will consume to render the named figure.

Required run — not produced on a laptop

Figure 1 (GPU/CPU crossover). *Hardware:* a single node with one CUDA-12-capable GPU with ≥ 40 GB HBM (e.g. NVIDIA A100 40 GB, H100 80 GB) and a ≥ 16 -core x86_64 host CPU (e.g. AMD EPYC 7763, Intel Xeon 8358). *Run command:*

Figure 1: GPU-vs-CPU Lanczos crossover

Figure 1: GPU-vs-CPU Lanczos wall time per iteration for the 1D Heisenberg chain at $S = 1/2$ as a function of Hilbert dimension. The expected behaviour is that LANCZOS_GPU crosses below LANCZOS (CPU 16-thread) above $d \sim 2.6 \times 10^5$ and keeps a 5–10 $\times$ advantage beyond $d \sim 10^7$.

```
python3 benchmarks/bench_all_backends.py \
    --build-dir build \
    --sizes 16 18 20 22 24 26 \
    --threads $(nproc) \
    --backends LANCZOS,LANCZOS_GPU \
    --output bench_gpu_cpu_crossover.json
python3 scripts/plot_bench_crossover.py \
    bench_gpu_cpu_crossover.json \
    --out paper/figures/gpu_cpu_crossover.pdf
```

Expected wall-clock: ~ 15 min on the listed hardware ($N = 24$ dominates with ~ 9 min).

421

Required run — not produced on a laptop

Figure 2 (distributed strong scaling). *Hardware:* an HPC cluster with ≥ 64 MPI ranks across at least 4 nodes, each node ≥ 256 GB DRAM, InfiniBand HDR or better, MPICH-4 or OpenMPI-5. Reference target: a Compute Canada *rorqual*-class allocation. *Run command:*

```
for R in 1 2 4 8 16 32 64; do
    mpiexec -n $R ./build/examples/ex05_mpi_distributed_lanczos \
        --N=28 --tolerance=1e-12 \
        --output=bench_strong_scaling_R${R}.json
done
python3 scripts/plot_strong_scaling.py \
    bench_strong_scaling_R*.json \
    --out paper/figures/dist_strong_scaling.pdf
```

Expected wall-clock: ~ 2 hours total, dominated by $R = 1$ (~ 45 min) and $R = 2$ (~ 30 min).

422

Figure 2: distributed Lanczos strong scaling

Figure 2: Strong-scaling efficiency of `distributed_lanczos` on the 1D Heisenberg chain at $N = 28$, full Hilbert space, fixed problem dimension $d = 2^{28} \approx 2.7 \times 10^8$. The Phase 8 batched-CGS2 reorthogonalisation (Section 4.6) is expected to push the parallel efficiency from ~ 0.45 (Phase 7 baseline, MGS) to ~ 0.75 at $R = 64$ ranks.

Required run — not produced on a laptop

Figure 3 (weak scaling). *Hardware:* an HPC cluster with ≥ 256 MPI ranks, ≥ 16 nodes, InfiniBand HDR, MPICH-4 or OpenMPI-5. *Run command:*

```
# (R, N) pairs chosen so d/R = 2^24 stays constant.
for pair in "1 24" "2 25" "4 26" "8 27" "16 28" "32 29" "64 30" "128 31" "256 32";
do
    R=${pair% *}; N=${pair## *}
    mpiexec -n $R ./build/examples/ex05_mpi_distributed_lanczos \
        --N=$N --tolerance=1e-10 \
        --output=bench_weak_R${R}_N${N}.json
done
python3 scripts/plot_weak_scaling.py \
    bench_weak_R*_N*.json \
    --out paper/figures/dist_weak_scaling.pdf
```

Expected wall-clock: ~ 3 hours, dominated by the $N = 32, R = 256$ point.

423

Required run — not produced on a laptop

Figure 4 (NCCL multi-GPU Lanczos). *Hardware:* a single node with 4 NVIDIA H100 80 GB SXM5 GPUs on NVLink (e.g. an HGX H100 8-GPU baseboard with the lower 4 GPUs bound). Reference target: `rorqual` login nodes (`rorqual-h100`). *Run command:*

```
for G in 1 2 4; do
    mpiexec -n $G --bind-to numa \
        ./build/ed_distributed_main \
        ./chain28 --method=LANCZOS_GPU_MPI \
        --tolerance=1e-12 \
```

424

Figure 3: distributed Lanczos weak scaling

Figure 3: Weak-scaling parallel efficiency of `distributed_lanczos` on the 1D Heisenberg chain holding $d/R = 2^{24}$ states per rank constant, varying N and R together so that the rank-local slab size stays at ~ 256 MB. Phase 8 batched CGS2 keeps the network round-trips per Lanczos iteration at 2 (one for α , one for β , plus 2 batched ones for reorth) regardless of R , giving an essentially flat weak-scaling curve out to $R = 256$.

```

--output=bench_multigpu_G${G}.json
done
python3 scripts/plot_multigpu.py \
    bench_multigpu_G*.json \
    --out paper/figures/multigpu_lanczos.pdf

```

Expected wall-clock: ~ 1 hour on $4 \times$ H100.

425

Required run — not produced on a laptop

Figure 5 (Phase 8 ablation). *Hardware:* 16 MPI ranks across 4 H100 nodes (≥ 4 ranks/node), `rorqual-h100`. *Run command:* build the toolkit at six git tags and run a fixed-size benchmark at each; the `benchmarks/bench_phase8_ablation.sh` script automates this.

```

bash benchmarks/bench_phase8_ablation.sh \
    --N=28 --ranks=16 \
    --tags=phase7-baseline,phase8-haloreuse,phase8-threadbudget,phase8-cgs2, \
    phase8-cublas-devptr,phase8-final \
    --output=bench_phase8_ablation.json
python3 scripts/plot_phase8_ablation.py \
    bench_phase8_ablation.json \
    --out paper/figures/phase8_ablation.pdf

```

Expected wall-clock: ~ 2 hours.

426

Figure 4: NCCL multi-GPU Lanczos

Figure 4: Multi-GPU strong scaling of `distributed_lanczos_gpu` on the 1D Heisenberg chain at $N = 28$, full Hilbert space, with NCCL halo exchange. Expected efficiency at 4 GPUs is ~ 0.85 on a single-node $4 \times$ H100 NVLink topology (the configuration we have correctness-locked, see Section 2.4).

Required run — not produced on a laptop

Figure 6 (ScaLAPACK auto-block-size). *Hardware:* 4 to 32 MPI ranks on an HPC node with MKL ScaLAPACK 2024 or netlib ScaLAPACK 2.2. *Run command:*

```
for N in 16 32 64 128; do
  for R in 4 8 16 32; do
    for blk in 8 16 32 64 128 auto; do
      if [ "$blk" = "auto" ]; then
        mpiexec -n $R ./build/ED ./chain${N} --method=SCALAPACK \
          --scalapack-block-size=auto
      else
        mpiexec -n $R ./build/ED ./chain${N} --method=SCALAPACK \
          --scalapack-block-size=${blk}
      fi
    done
  done
done | tee bench_scalapack_auto.log
python3 scripts/plot_scalapack_auto.py \
  bench_scalapack_auto.log \
  --out paper/figures/scalapack_auto.pdf
```

Expected wall-clock: ~ 4 hours total at $N = 128$, dominated by the $R = 4, m_b = 8$ point.

427

Required run — not produced on a laptop

Figure 7 (distributed FTLT thermodynamics). *Hardware:* 64 MPI ranks, ≥ 4 nodes, ≥ 256 GB DRAM per node. *Run command:*

```
mpiexec -n 64 ./build/ed_distributed_main \
  ./kagome32 --method=FTLM \
  --num-samples=200 --max-iter=200 \
```

428

Figure 5: Phase 8 ablation

Figure 5: Wall-clock-per-Lanczos-iteration ablation of the Phase 8 GPU and MPI optimisations on the same problem ($N = 28$ Heisenberg chain, $R = 16$ MPI ranks, single-GPU per rank, full reorthogonalisation). Bars (left to right) for: Phase 7 baseline; + persistent halo (Section 4.4); + `ThreadBudgetScope` on host code (Section 4.2); + batched CGS2 reorth (Section 4.6); + device-pointer-mode cuBLAS α (Section 4.5); + persistent ring-buffer pointer table (Section 4.4). Expected cumulative speedup at this scale: $\sim 2.4\times$ over Phase 7.

```
--temperature-grid=0.05:5.0:logarithmic:80 \
--output=ftlm_kagome32.h5
python3 scripts/plot_ftlm_thermo.py \
ftlm_kagome32.h5 \
--out paper/figures/dist_ftlm_kagome32.pdf
```

Expected wall-clock: ~ 8 hours.

429

430 7 Discussion and conclusions

431 We have described `exact_diagonalization.cpp`, an ED toolkit whose central design
 432 choice is to make the *(solver, fixed_sz, gpu, mpi, symm)* axes orthogonal at the parameter-
 433 bag level so that any algorithm in the dispatcher composes with any backend. This is
 434 mostly a software-engineering claim, but it has a measurable user-side payoff: the same
 435 Python script that computes a 12-site ground state on a laptop with one method swap
 436 becomes a 28-site distributed run on a 64-rank HPC allocation, and a GPU per-sector run
 437 on an H100 in between, without rewriting any basis-construction or post-processing logic.

438 The cross-cutting performance work (Phase 6.1 + Phase 8) closes the gap between
 439 the dispatcher’s flexibility and a hand-coded one-method ED kernel. On the CPU side,
 440 the SpMV+Lanczos pairing beats QuSpin’s `hamiltonian.dot+eigsh` by orders of magni-
 441 tude in the $d \in [4,096, 262,144]$ regime, and `XDiag.eigval0` on the same chain at every N
 442 tested. On the GPU and MPI sides the Phase 8 changes (batched CGS2 reorthogonalisa-
 443 tion; persistent halo, send/recv, and ring-buffer pointer-table buffers; device-pointer-mode

Figure 6: ScaLAPACK auto-block-size

Figure 6: ScaLAPACK `pzheevd` wall time as a function of N and BLACS block size, comparing the legacy fixed default $m_b = 64$ against the Phase 8 `scalapack.block_size.auto=true` heuristic. The auto path is expected to track the per- N minimum within $\sim 5\%$ across $N \in \{16, 32, 64, 128\}$ and 4–32 MPI ranks; the fixed default loses up to $\sim 2.5\times$ at the extremes.

cuBLAS scalars; ScaLAPACK auto-block-size) bring the distributed Lanczos and GPU Lanczos to bandwidth-limited operation. We expect, in the order-of-magnitude sense, the strong-scaling efficiency at $R = 64$ ranks to recover from ~ 0.45 to ~ 0.75 (Section 6.4), but verifying this requires the distributed runs sketched in Figs. 2 and 5.

Several extensions are punted to future work. A persistent `MPI_Alltoallv` halo via `MPI_Alltoallv_init` (MPI 4.0) would fold the halo plan creation cost into a one-time upfront amortisation, giving us a further $\sim 10\text{--}15\%$ on the distributed Lanczos at small d/R . A device-resident Hessenberg in `GPUKrylovSchur` (mirroring the Phase 8 fix in `GPULanczos`) and an event-fan-in stream pool in `GPUBlockKrylovSchur::blockMatVec` are the natural follow-ups on the GPU side. A parallel-HDF5 distributed disk-backed Krylov basis would push the symmetry-projected reachable N from ~ 32 to ~ 36 on a fat node. Finally, a distributed DSSF/SSSF implementation (currently gated on Mori-projection bookkeeping) would make finite- T dynamical structure factors at $N \geq 32$ reachable.

The package is released under the MIT licence at https://github.com/zhoub79/exact_diagonalization_clean. A versioned `CHANGELOG.md` and a per-release `CITATION.cff` (DOI to be assigned at the first tagged release) live at the repository root.

Code availability

The complete source code, test suite, examples, benchmark reproducers, and Sphinx/Doxygen documentation source are at https://github.com/zhoub79/exact_diagonalization_clean under the MIT licence. The release commit corresponding to this manuscript is the `phase-8-final` tag.

Figure 7: distributed FTLM thermodynamics

Figure 7: Specific heat $C(T)$ of the $N = 32$ kagome Heisenberg antiferromagnet computed by `distributed_ftlm` with 200 samples and $m = 200$ Lanczos vectors per sample, on 64 MPI ranks. Reference data overlaid from the published high- T series and the Sandvik QMC up to the sign-problem boundary.

Acknowledgements

We thank the developers of QuSpin [15,16], $\mathcal{H}\Phi$ [17], and XDiag [19] for setting the bar in this design space; the MPI Forum and the NVIDIA cuBLAS / cuSPARSE / cuSOLVER / NCCL teams for the building blocks; and the maintainers of the HDF group's HDF5 library [36,37] for the output-format substrate.

Funding information Funding sources to be filled in.

Author contributions Z.Z. designed and implemented the toolkit, wrote the test suite, and wrote the manuscript.

Competing interests The author declares no competing interests.

References

- [1] H.-S. Lin, *Exact diagonalization of quantum-spin models*, Physical Review B **42**(10), 6561 (1990), doi:10.1103/PhysRevB.42.6561.
- [2] H. Q. Lin and J. E. Gubernatis, *Exact diagonalization methods for quantum systems*, Computers in Physics **7**(4), 400 (1993), doi:10.1063/1.4823192.
- [3] A. W. Sandvik, *Computational studies of quantum spin systems*, AIP Conference Proceedings **1297**, 135 (2010), doi:10.1063/1.3518900.

- [4] A. M. Läuchli, *Numerical simulations of frustrated systems*, In C. Lacroix, P. Mendels and F. Mila, eds., *Introduction to Frustrated Magnetism*, vol. 164 of *Springer Series in Solid-State Sciences*, pp. 481–511. Springer, doi:10.1007/978-3-642-10589-0_18 (2011).
- [5] A. Wietek and A. M. Läuchli, *Sublattice coding algorithm and distributed memory parallelization for large-scale exact diagonalizations of quantum many-body systems*, *Physical Review E* **98**(3), 033309 (2018), doi:10.1103/PhysRevE.98.033309.
- [6] J. Jaklič and P. Prelovšek, *Lanczos method for the calculation of finite-temperature quantities in correlated systems*, *Physical Review B* **49**(7), 5065 (1994), doi:10.1103/PhysRevB.49.5065.
- [7] M. Aichhorn, M. Daghofer, H. G. Evertz and W. von der Linden, *Low-temperature Lanczos method for strongly correlated systems*, *Physical Review B* **67**(16), 161103 (2003), doi:10.1103/PhysRevB.67.161103.
- [8] A. Hams and H. De Raedt, *Fast algorithm for finding the eigenvalue distribution of very large matrices*, *Physical Review E* **62**(3), 4365 (2000), doi:10.1103/PhysRevE.62.4365.
- [9] S. Sugiura and A. Shimizu, *Thermal pure quantum states at finite temperature*, *Physical Review Letters* **108**(24), 240401 (2012), doi:10.1103/PhysRevLett.108.240401.
- [10] S. Sugiura and A. Shimizu, *Canonical thermal pure quantum state*, *Physical Review Letters* **111**(1), 010401 (2013), doi:10.1103/PhysRevLett.111.010401.
- [11] R. B. Lehoucq, D. C. Sorensen and C. Yang, *ARPACK Users’ Guide: Solution of Large-Scale Eigenvalue Problems with Implicitly Restarted Arnoldi Methods*, SIAM, doi:10.1137/1.9780898719628 (1998).
- [12] G. W. Stewart, *A Krylov–Schur algorithm for large eigenproblems*, *SIAM Journal on Matrix Analysis and Applications* **23**(3), 601 (2002), doi:10.1137/S0895479800371529.
- [13] K. Wu and H. Simon, *Thick-restart Lanczos method for large symmetric eigenvalue problems*, *SIAM Journal on Matrix Analysis and Applications* **22**(2), 602 (2000), doi:10.1137/S0895479898334605.
- [14] Y. Saad, *Numerical Methods for Large Eigenvalue Problems*, SIAM, 2 edn., doi:10.1137/1.9781611970739 (2011).
- [15] P. Weinberg and M. Bukov, *QuSpin: a Python package for dynamics and exact diagonalisation of quantum many-body systems part I: spin chains*, *SciPost Physics* **2**(1), 003 (2017), doi:10.21468/SciPostPhys.2.1.003.
- [16] P. Weinberg and M. Bukov, *QuSpin: a Python package for dynamics and exact diagonalisation of quantum many-body systems. Part II: bosons, fermions and higher spins*, *SciPost Physics* **7**(2), 020 (2019), doi:10.21468/SciPostPhys.7.2.020.
- [17] M. Kawamura, K. Yoshimi, T. Misawa, Y. Yamaji, S. Todo and N. Kawashima, *Quantum lattice model solver $\mathcal{H}\phi$* , *Computer Physics Communications* **217**, 180 (2017), doi:10.1016/j.cpc.2017.04.006.
- [18] B. Bauer, L. D. Carr, H. G. Evertz, A. Feiguin, J. Freire, S. Fuchs, L. Gamper, J. Gukelberger, E. Gull, S. Guertler et al., *The ALPS project release 2.0: open source software for strongly correlated systems*, *Journal of Statistical Mechanics: Theory and Experiment* **2011**(05), P05001 (2011), doi:10.1088/1742-5468/2011/05/P05001.

- [19] A. Wietek *et al.*, *XDiag: a high-performance exact diagonalization library for quantum many-body systems*, arXiv preprint **arXiv:2505.02901** (2025), doi:10.48550/arXiv.2505.02901.
- [20] NVIDIA Corporation, *NVIDIA cuBLAS library*, <https://docs.nvidia.com/cuda/cublas/>, Online; CUDA Toolkit 12.x (2024).
- [21] NVIDIA Corporation, *NVIDIA cuSPARSE library*, <https://docs.nvidia.com/cuda/cusparse/> (2024).
- [22] NVIDIA Corporation, *NVIDIA cuSOLVER library*, <https://docs.nvidia.com/cuda/cusolver/> (2024).
- [23] NVIDIA Corporation, *NVIDIA collective communication library (NCCL)*, <https://docs.nvidia.com/deeplearning/nccl/> (2024).
- [24] C. Lanczos, *An iteration method for the solution of the eigenvalue problem of linear differential and integral operators*, Journal of Research of the National Bureau of Standards **45**(4), 255 (1950), doi:10.6028/jres.045.026.
- [25] C. C. Paige, *The computation of eigenvalues and eigenvectors of very large sparse matrices*, Ph.D. thesis, University of London (1971).
- [26] E. R. Davidson, *The iterative calculation of a few of the lowest eigenvalues and corresponding eigenvectors of large real-symmetric matrices*, Journal of Computational Physics **17**(1), 87 (1975), doi:10.1016/0021-9991(75)90065-0.
- [27] A. V. Knyazev, *Toward the optimal preconditioned eigensolver: Locally Optimal Block Preconditioned Conjugate Gradient method*, SIAM Journal on Scientific Computing **23**(2), 517 (2001), doi:10.1137/S1064827500366124.
- [28] E. Anderson, Z. Bai, C. Bischof, S. Blackford, J. Demmel, J. Dongarra, J. Du Croz, A. Greenbaum, S. Hammarling, A. McKenney and D. Sorensen, *LAPACK Users' Guide*, SIAM, 3 edn., doi:10.1137/1.9780898719604 (1999).
- [29] L. S. Blackford, J. Choi, A. Cleary, E. D'Azevedo, J. Demmel, I. Dhillon, J. Dongarra, S. Hammarling, G. Henry, A. Petitet, K. Stanley, D. Walker *et al.*, *ScaLAPACK Users' Guide*, SIAM, doi:10.1137/1.9780898719642 (1997).
- [30] J. Choi, J. Demmel, I. Dhillon, J. Dongarra, S. Ostrouchov, A. Petitet, K. Stanley, D. Walker and R. C. Whaley, *ScaLAPACK: a portable linear algebra library for distributed memory computers — design issues and performance*, Computer Physics Communications **97**(1–2), 1 (1996), doi:10.1016/0010-4655(96)00017-3.
- [31] H. Mori, *Transport, collective motion, and brownian motion*, Progress of Theoretical Physics **33**(3), 423 (1965), doi:10.1143/PTP.33.423.
- [32] E. Dagotto, *Correlated electrons in high-temperature superconductors*, Reviews of Modern Physics **66**(3), 763 (1994), doi:10.1103/RevModPhys.66.763.
- [33] Å. Björck, *Numerics of Gram–Schmidt orthogonalization*, Linear Algebra and its Applications **197–198**, 297 (1994), doi:10.1016/0024-3795(94)90493-6.
- [34] L. Giraud, J. Langou, M. Rozložník and J. van den Eshof, *Rounding error analysis of the classical Gram–Schmidt orthogonalization process*, Numerische Mathematik **101**(1), 87 (2005), doi:10.1007/s00211-005-0615-4.

- 565 [35] MPI Forum, *MPI: A message-passing interface standard, version 4.0*, <https://www.mpi-forum.org/docs/> (2021).
566
- 567 [36] M. Folk, G. Heber, Q. Koziol, E. Pourmal and D. Robinson, *An overview of the HDF5*
568 *technology suite and its applications*, Proceedings of the EDBT/ICDT Workshop on
569 Array Databases pp. 36–47 (2011), doi:10.1145/1966895.1966900.
- 570 [37] The HDF Group, *Hierarchical data format, version 5*, <https://www.hdfgroup.org/HDF5/>
571 (1997-2024).